

FschoolAI — Product Requirements Document (PRD)

Version: 1.0

Date: June 22, 2026

Author: Vincent Yang, FschoolAI

Audience: Engineering team — Tencent engineer, Bytedance engineer, Aryan, Ryan, Vincent

1. Product Vision

FschoolAI is the **personal academic intelligence** for every student. It is not a generic AI chatbot. It is not a homework helper. It is a persistent, living brain that knows each student individually — their learning style, their knowledge gaps, their deadlines, their stress level, their history — and uses that knowledge to help them from where they are to where they need to be.

The core principle is: **the AI adapts to the student, not the student to the AI.**

Every student is unique. A student at Carleton studying engineering has different gaps, different pressures, and different ways of learning than a student at UCLA studying pre-med. FschoolAI treats each student as a unique individual, not a user type.

At signup, a **student second brain** is created for each user. This brain is theirs. It grows with every session — every question asked, every assignment completed, every lecture attended. When NeuroAGI hardware launches, the student claims their brain on the device. Until then, the brain lives in the cloud and is accessible through FschoolAI.

2. Target Users

FschoolAI is built for university and college students globally, with initial focus on North America. The product must be immediately useful to an international student who is not a native English speaker, a first-generation university student with no academic support network, and a high-achieving student who wants to go beyond what their institution provides.

The product must be so useful that students are willing to pay for it personally — not wait for their school to buy it.

3. Core Architecture

3.1 The Student Brain

Every student has a brain object stored in the NeuroAGI brain layer. This is a persistent, structured knowledge graph about the student. It is the foundation that every agent reads from and writes to.

```
StudentBrain {
  student_id: string
  learning_style: "visual" | "auditory" | "reading" | "kinesthetic" | "mixed"
  knowledge_nodes: KnowledgeNode[] // what the student knows and gaps
  course_context: CourseContext[] // current courses, assignments,
  stress_level: float (0.0-1.0) // inferred from behaviour patterns
  session_history: Session[] // all past interactions
  performance_signals: Signal[] // quiz scores, time-on-task, completion
  preferences: Preferences // communication style, response time
  upcoming_deadlines: Deadline[] // from Canvas sync
  created_at: timestamp
  last_updated: timestamp
}
```

3.2 Agent Architecture

Every agent follows the same three-step pattern:

```
Step 1: context = brain.read(student_id)
Step 2: result = agent_logic(user_input, context)
Step 3: brain.write(student_id, signal)
```

Agents do not store state themselves. All state lives in the brain. This means any agent can be replaced or upgraded without losing the student's history.

3.3 Phase 1 — Mock Brain (Build Now)

During Phase 1, agents use a local mock context object instead of calling the NeuroAGI brain API. This allows all agents to be built and tested immediately without waiting for the brain layer.

```
{
  "student_id": "mock_001",
  "learning_style": "visual",
  "knowledge_gaps": ["integration by parts", "organic mechanisms"],
  "stress_level": 0.6,
  "upcoming_deadlines": [
    { "course": "CHEM 201", "assignment": "Lab Report 3", "due": "2026-06"
  },
  "preferred_language": "en",
  "session_count": 14
}
```

3.4 Phase 2 — Live Brain (After Ryan's API is ready)

Replace the mock context with:

```
const context = await brain.read(student_id)
// ... agent logic ...
await brain.write(student_id, signal)
```

No other change to any agent is required.

4. Agent Specifications

Agent 1 — Reggie (Orchestrator)

Owner: Vincent

Environment: All — Reggie is the entry point for every student interaction

Priority: P0 — must be built first, all other agents depend on it

What it does: Reggie receives the student's message, reads the brain context, decides which specialist agent to call, passes the message and context to that agent, and returns the response to the student. Reggie is the router and the face of FschoolAI.

Input: Student message (text, voice, image)

Output: Response from the appropriate specialist agent

Routing logic:

Student message type	Routes to
“Explain this concept” / “I don’t understand X”	Tutor Agent
“What do I have due?” / “Show my schedule”	Canvas Agent + Planner Agent
“Help me study for my exam”	Exam Mode Agent
“Make a study plan for this week”	Planner Agent
“Summarise this lecture”	Lecture Agent
“Find me resources on X”	Library Agent
“I’m stressed / overwhelmed”	Intervention Agent
“Translate this”	Audio Agent
Ambiguous	Reggie asks one clarifying question, then routes

Brain signals written after each session:

- `session_type` : what kind of help was requested
- `session_duration` : how long the session lasted
- `topics_discussed` : list of subjects covered

Agent 2 — Canvas Agent

Owner: Aryan

Environment: Canvas LMS integration

Priority: P0 — foundational data source for all other agents

What it does: Connects to the student’s Canvas account via OAuth. Syncs all courses, assignments, due dates, grades, and syllabus documents. Stores this data in FschoolAI’s Supabase database. Alerts the student about upcoming deadlines and missing work.

Canvas data pulled:

- All enrolled courses (current semester)
- All assignments per course (title, due date, points, submission status)

- All grades (current score, letter grade, course average)
- Syllabus documents (PDF/HTML)
- Announcement feed
- Professor contact information

Sync schedule: Every 6 hours automatically. On-demand when student opens FschooAI.

Alerts generated:

- Assignment due in less than 24 hours — push notification
- Assignment due in less than 72 hours — in-app banner
- Grade posted — in-app notification
- Missing assignment — weekly summary

Brain signals written:

- `course_list` : current courses
- `upcoming_deadlines` : next 14 days of assignments
- `grade_summary` : current standing per course
- `stress_signal` : if 3+ assignments due within 48 hours → `stress_level` += 0.2

Note for Aryan: Canvas uses OAuth 2.0. The student authorises FschooAI to read their Canvas data. FschooAI never stores the Canvas password. The Canvas API token is stored encrypted in Supabase.

Agent 3 — Tutor Agent

Owner: Tencent engineer

Environment: Chat / Study Session

Priority: P0 — core product, primary reason students pay

What it does: This is the heart of FschooAI. The Tutor Agent answers academic questions, explains concepts, identifies what the student does not understand, and adapts its explanation style to the student's learning profile. It does not just answer — it teaches.

Key behaviours:

Adaptive explanation: The Tutor Agent reads the student's learning style from the brain and adjusts its response format accordingly.

Learning style	Response format
Visual	Diagrams described in text, tables, step-by-step with visual structure
Auditory	Conversational, narrative, “imagine you are explaining to a friend”
Reading	Dense text, definitions, citations, structured paragraphs
Kinesthetic	Examples first, then theory, “try this yourself” prompts

Knowledge gap detection: When the student asks a question, the Tutor Agent identifies whether the question reveals a deeper gap. If the student asks “why does integration by parts work?”, the agent checks whether they understand the product rule first. If not, it teaches the prerequisite before answering the original question.

Socratic mode: For questions that are clearly homework or exam questions, the Tutor Agent does not give the answer directly. It asks guiding questions that lead the student to the answer themselves. This is configurable — the student can turn off Socratic mode if they just want the answer.

Simplification levels: Content must be adjustable to the student's level. International students and students who are not native English speakers need simpler language. The agent detects this from the student's communication style and adjusts automatically.

Input: Student question (text or image of problem)

Output: Explanation, worked example, follow-up question, or resource recommendation

Brain signals written:

- `topic_studied` : subject and concept
- `confusion_detected` : boolean — did the student express confusion?
- `gap_identified` : specific knowledge gap found
- `session_quality` : inferred from follow-up questions and engagement
- `explanation_format_used` : which format was used, for future optimisation

Agent 4 — Planner Agent

Owner: Bytedance engineer

Environment: Study Planner / Dashboard

Priority: P1

What it does: Builds personalised weekly study schedules based on the student's deadlines, available time, subject difficulty, and current stress level. The plan is not generic — it is specific

to this student's brain context.

Planning inputs (all from brain context):

- Upcoming deadlines (from Canvas Agent)
- Current stress level
- Knowledge gaps (which subjects need more time)
- Historical study patterns (when does this student actually study?)
- Student's stated available hours

Plan output format:

```
Monday June 23:  
  9:00-10:30  CHEM 201 – Stereochemistry review (gap identified)  
  14:00-15:00 MATH 204 – Practice integration by parts (gap identified)  
  19:00-20:00 Essay outline for ENGL 301 (due June 26)  
  
Tuesday June 24:  
  ...
```

Adaptive replanning: If the student misses a study block, the Planner Agent detects this (no activity during scheduled time) and automatically reschedules the missed work into the next available slot. It does not nag — it silently adjusts.

Brain signals written:

- `study_plan_created` : date and week covered
- `plan_adherence` : percentage of planned blocks completed
- `reschedule_count` : how many times the plan was adjusted

Agent 5 — Lecture Agent

Owner: Aryan

Environment: Chrome Extension / In-class

Priority: P1

What it does: Captures lecture audio through the Chrome extension, transcribes it in real time, generates a structured summary, identifies key concepts, and creates flashcards and quiz

questions from the lecture content. The student gets a complete lecture package within minutes of the lecture ending.

Lecture package output:

- Full transcript (searchable)
- Summary (3–5 key points)
- Concept list (terms defined)
- Flashcard set (auto-generated)
- 5 quiz questions (multiple choice + short answer)
- Connections to existing brain knowledge (“This concept relates to what you studied last week in CHEM 201”)

Note for Aryan: The Chrome extension captures audio from the student’s microphone (in-class) or from a browser tab (online lecture). Audio is processed via Whisper. The transcript and summary are stored in Supabase and linked to the student’s brain.

Brain signals written:

- `lecture_attended` : course, date, duration
 - `concepts_introduced` : list of new concepts from this lecture
 - `lecture_summary_generated` : boolean
-

Agent 6 — Library Agent

Owner: Bytedance engineer

Environment: Resource Library

Priority: P2

What it does: When the student needs resources — textbook chapters, academic papers, YouTube explanations, practice problems — the Library Agent finds and curates them. It does not return a generic Google search. It returns resources matched to the student’s learning style, current knowledge level, and specific gap.

Search behaviour:

- Reads the student’s learning style from brain context
- Reads the specific knowledge gap being addressed
- Returns 3–5 resources ranked by relevance and quality
- Includes a one-sentence explanation of why each resource was chosen

Resource types:

- Khan Academy videos (visual learners)
- Academic papers (reading learners)
- Practice problem sets (kinesthetic learners)
- Podcast episodes / audio explanations (auditory learners)
- YouTube explanations (all types)

Brain signals written:

- `resource_recommended` : resource type and topic
 - `resource_clicked` : whether the student opened the resource
-

Agent 7 — Exam Mode Agent

Owner: Vincent

Environment: Exam Preparation

Priority: P1

What it does: When the student has an exam coming up, Exam Mode Agent creates a personalised exam preparation plan. It identifies the highest-priority topics based on the student's knowledge gaps, generates practice questions, simulates exam conditions, and tracks progress toward exam readiness.

Exam prep flow:

1. Student says "I have an exam in CHEM 201 in 3 days"
2. Agent reads brain context: knowledge gaps, past quiz performance, time available
3. Agent generates a 3-day prep plan prioritising the student's weakest areas
4. Agent generates 20 practice questions (mix of difficulty levels)
5. Student completes practice questions — agent evaluates answers
6. Agent updates the plan based on performance: if student struggles with topic X, add more practice on X
7. Day before exam: final review summary — "You are strong on A and B. Focus your last hour on C."

Brain signals written:

- `exam_prep_started` : course and exam date
-

- `practice_questions_completed` : count and score
 - `exam_readiness_score` : 0–100 estimated readiness
 - `weak_areas_at_exam_time` : final gap list before exam
-

Agent 8 — Intervention Agent

Owner: Vincent

Environment: Background monitor

Priority: P1

What it does: Runs silently in the background. Monitors the student’s stress signals, deadline proximity, and engagement patterns. When it detects a student who is overwhelmed, falling behind, or disengaged, it intervenes proactively — not with a generic notification, but with a specific, helpful action.

Intervention triggers:

Signal	Threshold	Intervention
Stress level	> 0.8	“You have 3 assignments due in 48 hours. Want me to build a plan?”
No study activity	3+ days before deadline	“Your essay is due in 3 days and I haven’t seen you work on it. Want to start now?”
Repeated confusion on same topic	3+ sessions	“You’ve asked about integration by parts 3 times. Let me try a different explanation.”
Grade drop	> 10% below course average	“Your CHEM 201 grade dropped this week. Want to review what was covered?”
Late night pattern	Study sessions after 1am for 3+ nights	“You’ve been studying late. A 20-minute review now is more effective than 2 hours at midnight.”

Brain signals written:

- `intervention_triggered` : type and trigger
 - `intervention_accepted` : boolean — did the student engage with the intervention?
-

Agent 9 — Audio Agent

Owner: Aryan

Environment: Audio / Multilingual

Priority: P2

What it does: Handles all audio-related tasks. Transcribes lecture recordings, translates content into the student's preferred language, converts text explanations to audio for auditory learners, and processes voice input from the student.

Capabilities:

- Speech-to-text (Whisper) — transcribe any audio file or live recording
- Text-to-speech — read explanations aloud
- Translation — translate lecture content, study materials, or AI responses into the student's language
- Language detection — automatically detects the student's preferred language from their messages

Supported languages (Phase 1): English, Mandarin Chinese, Hindi, French, Spanish, Arabic

Brain signals written:

- `preferred_language` : detected from student's messages
 - `audio_sessions` : count of audio-mode interactions
-

Agent 10 — Office Hours Agent

Owner: Tencent engineer

Environment: Office Hours Preparation

Priority: P2

What it does: Helps the student prepare for and follow up on professor office hours. Before office hours, it generates a list of specific questions based on the student's knowledge gaps. After office hours, it helps the student record what was discussed and updates the brain with new knowledge.

Pre-office hours flow:

1. Student says "I have office hours with Professor Chen in 30 minutes"
2. Agent reads brain context: current gaps in the relevant course
3. Agent generates 3–5 specific, well-formed questions to ask the professor
4. Agent briefs the student: "Here is what you should ask and why"

Post-office hours flow:

1. Student says “Office hours just ended”
2. Agent asks: “What did you learn? What was clarified?”
3. Student describes what happened
4. Agent updates the brain: gaps closed, new concepts added

Brain signals written:

- `office_hours_attended` : course and professor
 - `questions_prepared` : list of questions generated
 - `gaps_closed` : knowledge gaps resolved in the session
-

Agent 11 — Calendar Agent

Owner: Bytedance engineer

Environment: Calendar Integration

Priority: P2

What it does: Integrates with Google Calendar and Apple Calendar. Reads the student’s existing schedule (classes, work, commitments) and uses this to make the Planner Agent’s study plans realistic. It also writes study blocks back to the student’s calendar.

Calendar reads:

- Class schedule (recurring events)
- Work shifts
- Social commitments
- Existing study blocks

Calendar writes:

- Study blocks generated by Planner Agent
- Exam dates (from Canvas)
- Assignment deadlines (from Canvas)

Brain signals written:

- `available_hours_per_day` : calculated from calendar gaps
-

- `calendar_connected` : boolean
-

Agent 12 — Reflection Agent

Owner: Ryan (NeuroAGI)

Environment: Background — runs nightly

Priority: P1

What it does: This agent runs every night at 2am for each student. It reviews the day's interactions, synthesises patterns, updates the brain graph, decays stale knowledge nodes, and prepares the brain for the next day. This is the agent that makes the brain smarter over time.

Nightly tasks:

- Review all signals written by other agents during the day
- Update knowledge gap confidence scores (did the student show improvement?)
- Decay old knowledge nodes (concepts not revisited in 30+ days lose confidence)
- Identify new patterns (e.g., "student consistently struggles on Mondays after weekend")
- Update stress level based on weekly trajectory
- Generate a "brain health summary" for the student (optional, shown in dashboard)

Note: This agent lives inside NeuroAGI, not FschooIAI. Ryan owns it. FschooIAI agents do not call it directly — it runs automatically on the NeuroAGI brain layer.

Agent 13 — Terminal Agent

Owner: Vincent

Environment: Developer / Power User Mode

Priority: P3

What it does: For advanced students who want to query their own brain directly, run custom workflows, or inspect their data. Think of it as a command-line interface to the student's brain.

Example commands:

```
> show my knowledge gaps in CHEM 201
> what topics have I studied this week?
> export my study history as CSV
> set my learning style to visual
> show my stress level trend this month
```

Brain signals written:

- `terminal_commands_used` : list of commands executed
-

5. User Flows

5.1 Onboarding Flow

The onboarding is the “identity card” session — a one-on-one setup with FschooIAI that builds the student’s initial brain profile.

Step 1 — Account creation Student signs up with email or Google. A blank brain object is created with their `student_id`.

Step 2 — Canvas connection Student connects their Canvas account via OAuth. Canvas Agent syncs all courses, assignments, and grades. The brain is populated with course context and upcoming deadlines.

Step 3 — Learning style assessment Reggie asks 5 quick questions to determine the student’s learning style. These are conversational, not a formal test. Example: “When you are trying to understand something new, do you prefer to see a diagram, hear an explanation, read about it, or try it yourself?”

Step 4 — First brain summary FschooIAI shows the student their initial brain: “Here is what I know about you so far. You are taking 4 courses. Your next deadline is in 2 days. I think you learn best visually. Is this right?”

Step 5 — First interaction Reggie asks: “What do you want to work on today?” The student is now in the product.

5.2 Daily Use Flow

Student opens FschooIAI

- Reggie shows a personalised daily brief:
"Good morning. You have 2 assignments due this week.
You have a study block for CHEM 201 at 2pm.
Your exam readiness for MATH 204 is 62% – want to practice?"

Student asks a question **or** picks a task

- Reggie routes **to** the appropriate agent
- Agent **reads** brain context
- Agent responds
- Agent writes signal **to** brain

End of session

- Reflection Agent (nightly) synthesises the **day**
- Brain **is** updated
- Tomorrow's **brief is prepared**

5.3 Exam Preparation Flow

Student: "I have a CHEM 201 exam on Friday"

- Reggie routes **to** Exam Mode Agent
- Exam Mode Agent reads brain: gaps **in** stereochemistry **and** reaction mechanisms
- Agent creates **3**-day prep plan
- Day **1**: stereochemistry practice (student's weakest area)
- Day **2**: reaction mechanisms + mixed practice
- Day **3**: full mock exam + review
- **Each** day: agent evaluates student's answers, updates readiness score
- Thursday night: "You are at **78% readiness**. Focus on reaction mechanisms"

6. Data Model

6.1 Core Tables (Supabase)

students

```
id          uuid PRIMARY KEY
email       text UNIQUE NOT NULL
name        text
canvas_token text (encrypted)
created_at  timestamp
last_active timestamp
```

brain_context (Phase 1 mock — later replaced by NeuroAGI API)

```
id          uuid PRIMARY KEY
student_id  uuid REFERENCES students(id)
learning_style text
stress_level float
knowledge_gaps jsonb
preferences jsonb
updated_at  timestamp
```

courses

```
id          uuid PRIMARY KEY
student_id  uuid REFERENCES students(id)
canvas_course_id text
name        text
professor   text
semester    text
```

assignments

```
id                uuid PRIMARY KEY
course_id         uuid REFERENCES courses(id)
canvas_assignment_id text
title             text
due_date          timestamp
points_possible   float
points_earned     float
submitted         boolean
```

sessions

```
id                uuid PRIMARY KEY
student_id        uuid REFERENCES students(id)
agent_used        text
input_text        text
output_text       text
topics            text[]
duration_seconds  int
created_at        timestamp
```

brain_signals

```
id                uuid PRIMARY KEY
student_id        uuid REFERENCES students(id)
signal_type       text
signal_data       jsonb
created_at        timestamp
```

7. Technical Stack

Layer	Technology
Frontend	React 19, Tailwind CSS 4, shadcn/ui
Mobile	React Native (Phase 2)
Backend	Supabase (database + auth + storage)
Brain layer	NeuroAGI API (Phase 2) / local mock JSON (Phase 1)
LLM	GPT-4o via OpenAI API (Phase 1)
Speech-to-text	OpenAI Whisper
Canvas integration	Canvas LMS REST API + OAuth 2.0
Calendar integration	Google Calendar API + Apple CalDAV
Chrome extension	Manifest V3, React
Hosting	Manus (FschoolAI frontend)

8. Build Order and Ownership

The build order is designed so no engineer blocks another. Ryan’s brain mock is available from Day 1 so all agents can develop against it.

Week	What gets built	Owner
Week 1	Brain mock JSON + <code>brain.read()</code> / <code>brain.write()</code> stub	Ryan
Week 1	Canvas Agent — OAuth + sync	Aryan
Week 1	Reggie — basic routing	Vincent
Week 2	Tutor Agent — core explanation logic	Tencent engineer
Week 2	Planner Agent — study schedule generation	Bytedance engineer
Week 2	Lecture Agent — transcription + summary	Aryan
Week 3	Exam Mode Agent	Vincent
Week 3	Intervention Agent	Vincent
Week 3	Library Agent	Bytedance engineer
Week 4	Audio Agent	Aryan
Week 4	Office Hours Agent	Tencent engineer
Week 4	Calendar Agent	Bytedance engineer
Week 5	Reflection Agent (NeuroAGI)	Ryan
Week 5	Terminal Agent	Vincent

Week	What gets built	Owner
Week 6	Replace mock brain with live NeuroAGI API	Ryan + all

9. Non-Functional Requirements

Response time: Every agent must respond within 3 seconds for standard queries. Lecture transcription and exam prep plan generation may take up to 10 seconds with a loading indicator.

Privacy: Student data is never used to train models. Canvas tokens are stored encrypted. Brain data belongs to the student — they can export or delete it at any time.

Language: All agents must support English by default. Mandarin Chinese support is required for Phase 1 (Chinese student market). Additional languages via Audio Agent in Phase 2.

Offline mode: Core brain context is cached locally. Students can view their study plan and upcoming deadlines without internet. Active agent sessions require internet.

Accessibility: All UI must meet WCAG 2.1 AA. Voice input must be available for all agent interactions.

10. Success Metrics

Metric	Target (Month 3)
Daily active users	500+
Session length	> 8 minutes average
Canvas connections	> 70% of users connect Canvas
Agent interactions per session	> 2 agents used per session
Student-reported grade improvement	> 40% of users report improvement
Retention (Day 30)	> 45%
NPS	> 50

11. Out of Scope (Phase 1)

The following are explicitly out of scope for Phase 1 and should not be built until Phase 2:

- NeuroAGI hardware integration (Neural Card)
- School/institution-facing dashboard
- Professor tools
- Payment processing (subscription billing)
- Native mobile app (iOS/Android)
- Social features (study groups, leaderboard)
- EducAI integration

This document is the source of truth for FschooIAI Phase 1 engineering. Any questions, contact Vincent Yang.